

[← Go Back](#)

Hardware

Edge Control



Tutorials

[Edge Control User Manual](#)[Smart Farm Irrigation System Using Arduino® Edge Control](#)[LPWAN Irrigation System Using Arduino® Edge Control](#)[Connecting and Controlling a Motorized Ball Valve](#)

Home / Hardware / Edge Control /
[Smart Farm Irrigation System Using Arduino® Edge Control](#)

Smart Farm Irrigation System Using Arduino® Edge Control

This application note describes how to control a four zones irrigation system using the Edge Control and the Arduino Cloud.

Author · Christopher Mendez · Last revision · 02/28/2025

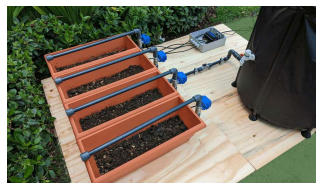
ON THIS PAGE

Introduction

[Goals](#)[Hardware and Software Requirements](#) +[Smart Irrigation System Setup](#)[Smart Irrigation System Overview](#) +[Full Smart Irrigation System Example](#) +[Conclusion](#) +[Trademark Acknowledgments](#)

Introduction

Agriculture has always been a key part of human development. Humanity is constantly growing and the demand for more efficient, profitable, and green agriculture is one of the challenges that the industry has been working on during the last years.



Application Note Overview.
Each pot represents one individual irrigation zone capable of watering a crop field

Smart farming is one of the most important actors in this revolution and is more accessible today than

[Help](#)

control capabilities, sensor data analysis and Cloud connectivity, Arduino Edge Control is perfect for managing agriculture applications.

The shown application note is intended to replicate a scaled smart farming application, that can be implemented on real agriculture fields using the same hardware and firmware.

Goals

The goal of this application note is to showcase a smart farming irrigation system using a combination of an Edge Control, an MKR WiFi 1010, and the Arduino Cloud. The project's objectives are the following:

- Independently control four irrigation zones using motorized ball valves.

- Get water from a smart tank with water level monitoring.

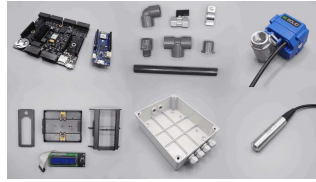
- Program irrigation timers from remote through Arduino Cloud by using Wi-Fi® connectivity.

- Manually activate irrigation from Arduino Cloud through dedicated widgets.

- Monitor average irrigation time and water consumption on dedicated charts on Arduino Cloud.

- Plan irrigation according to the weather forecast, by including an API-based weather station in Arduino Cloud.

Hardware and Software Requirements



Project main hardware and materials

Hardware Requirements

Arduino Edge Control

Arduino MKR WiFi 1010

Arduino Edge Control Enclosure Kit

Differential pressure Liquid Level Sensor - 4-20 mA (0-1 meter) Submersible.

2-Wires Motorized ball Valves (3-Wires version are also supported) (x4)

12 VDC power supply (x1)

3.4 meters of 15 mm PVC pipes (x1)

15 mm PVC TEE pipes (x3)

15 mm PVC elbow (x8)

15 mm Manual Valve (x1)

15 mm PVC caps (x4)

15 mm PVC male adapters (x11)

15 mm wall pipe brackets (x7)

Rectangular planters (x4)

DIN rail (x1)

Cable glands (x6)

6 meters of duplex cable AWG 18 (x1)

Electrical Register Box (x1)

Water Tank (x1)

Software Requirements

[Arduino IDE 1.8.10+](#), [Arduino IDE 2](#), or [Arduino Cloud Editor](#).

If you are going to use an offline Arduino IDE, you must

```
Arduino_JSON , ArduinoJson ,  
ArduinoHttpClient and  
Arduino_ConnectionHandler .
```

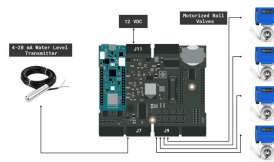
You can install them through the Arduino IDE Library Manager.

The [Smart Irrigation System Arduino Sketches](#).

[Arduino Create Agent](#) to provision the MKR WiFi 1010 on the Arduino Cloud.

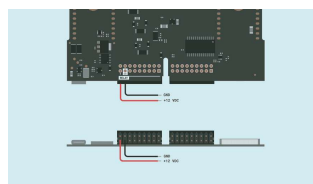
Smart Irrigation System Setup

The electrical connections of the intended application are shown in the diagram below:



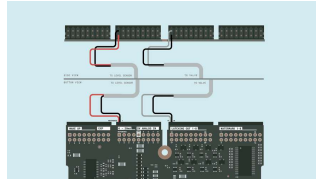
Electrical connections of the irrigation system

The Edge Control board will be powered with an external 12 VDC power supply connected to BATT+ and GND of J11 respectively.



Connecting the power supply

The four motorized ball valves will be connected to the Edge Control Latching outputs of J9 connector from OUT0 to OUT6.



2-Wires valve and 4-20 mA sensor connection detail



You can also use 3-Wires motorized valves without any changes in the code. See this [guide](#) for reference.

The water level transmitter will be connected to the +19 V reference and the 4-20 mA input number one of the J7 connector.



Project physical connections

Smart Irrigation System Overview

The irrigation system works as a whole: it integrates the level measurement and the activation of the valves, done by the Edge Control, with the Cloud communication, using the MKR WiFi 1010.

The Edge Control is responsible for keeping track of the time with its integrated real-time clock (RTC), in order to schedule the use of the valves and know when a day has passed. In addition, the same setup is able to:

level transmitter to calculate its consumption

Control an LCD screen where the status of the valves and timers are shown. In addition, the LCD push button can be used to manually activate the valves.

Moreover, the MKR WiFi 1010 is used to implement the communication between the setup and the Cloud. It notifies the Edge Control of any changes in the Cloud to activate, deactivate or configure a timer to control the valves. In addition, it reports the values of the Edge Control sensors on the Cloud. The communication between both devices is done leveraging the I2C communication protocol.



Smart irrigation system
with Edge Control

Valves Control

If a valve is activated from the Cloud, the message "opening valve" will appear on the LCD screen when the valve gets opened. If the valve is activated by a Cloud timer, the display will show the same message, including a countdown of the remaining irrigation time. The working time of the valves is monitored and reported on the Cloud to enable an efficient visualization of the average daily use.

At every system startup, the current amount of water is measured and saved. In this way, any decrease in it is counted as consumed water. The water use is reset daily to be able to monitor the average daily use on a dedicated Arduino Cloud chart.

Weather Forecast Consideration

The system is capable of knowing the weather forecast thanks to the use of the MKR WiFi 1010 and the OpenWeather API. If the rain probability gets greater than 90%, the automatic watering timers will be ignored and the smart irrigation will not start. However, it will be possible to continue irrigating manually if the user wishes to.

Arduino Edge Control Code

Let's go through some important code sections to make this application fully operative; starting with the required libraries:

`Arduino_EdgeControl.h` will enable the support for the Edge Control peripherals; install it by searching for it on the Library Manager.

`Wire.h` will enable the I2C communication between the Edge Control, the MKR WiFi 1010 and the other peripherals. It is included in the Board Support Package (BSP) of the Edge Control.

There are two headers included in the project code able to handle some helper functions and structures:

`SensorValues.hpp` handles the shared variables between

`helpers.h` handles the real-time clock (RTC) functions to retrieve the local date and time.

This code's section also contains the defined structure to handle the number of enclosure button taps to control each valve manually.

```
1 #include "Helpers.h"
2 #include <Arduino_EdgeCon
3 #include <Wire.h>
4 #include "SensorValues.hp
5
6 // The MKR1 board I2C add
7 #define EDGE_I2C_ADDR 0x6
8
9 /** UI Management **/
10 // Button statuses
11 enum ButtonStatus : byte
12     ZERO_TAP,
13     SINGLE_TAP,
14     DOUBLE_TAP,
15     TRIPLE_TAP,
16     QUAD_TAP,
17     FIVE_TAP,
18     LOT_OF_TAPS
19 };
```

In order to save energy and resources, the Edge Control has different power lines that must be enabled to power the different internal and external peripherals. In this case, the 3.3 V, 5 V, Battery, MKR1 slot, and the +19 V reference for the 4-20 mA sensor's current loop need to be enabled. To handle all the I/O, the I/O Expander together with the Enclosure Kit LCD and the sensors inputs need to be initialized.

With the `setSystemClock` function, a starting date reference for the real-time clock is defined. If you need to configure the RTC time with your time zone use the commented


```
RealTimeClock.setEpoch(<Your  
region unixTime>)
```

, replacing the parameter with your region unix time in seconds.

You just need to set the RTC once and make sure to have a

i CR2032 3V battery in the Edge Control holder to maintain the RTC configurations.

```
1  /**
2   Main section setup
3  */
4  void setup() {
5   EdgeControl.begin();
6   Wire.begin();
7   delay(500);
8   Serial.begin(115200);
9
10  Serial.println("Init k
11
12  // Enable power lines
13  Power.enable3V3();
14  Power.enable5V();
15  Power.on(PWR_3V3);
16  Power.on(PWR_VBAT);
17  Power.on(PWR_MKR1);
18  delay(5000);    // wai
19  Power.on(PWR_19V);
20
21  // Init Edge Control ]
22  Serial.print("IO Expar
23  if (!Expander.begin())
24   Serial.println("fail
25   Serial.println("Plea
26   Serial.println("via
27  } else Serial.println(
28
```

The Edge Control will check the number of button taps for the valve's manual control and handle the right action to do through the use of a switch case statement.

The `updateSensors()` function handles the update of system

the Cloud and retrieves online changes to maintain synchrony.

To measure the water level, a 4-20 mA (0 to 1 meter) sensor can be used, but you need to express that current information in centimeters first. The Edge Control converts the current from the sensor loop into a voltage by using an internal 220 ohms resistor, which is read by the internal ADC. To convert this voltage back to a current value, the following equation from a 4-20 mA sensor can be used:

$$y = 16x + 4$$

Where:

First we solve for x, having the formula: $x = (y - 4) / 16$ where x is in meters.

To be able to work in centimeters, you can multiply the expression by 100:

$$x = (y - 4) * (100 / 16)$$

Eventually, you can simplify the expression as:

$$x = (y - 4) * 6.25$$

This is a brief explanation of the mathematical expression used inside the sketch to convert the original sensor value voltage into centimeters:

```
float w_level =  
((voltsReference / 220.0 *  
1000.0) - 4.0) * 6.25;
```

In order to monitor the daily trend of the valve's active time on Arduino Cloud, the accumulated time is reset each day at midnight.

The `valvesHandler()` function activates, deactivates and keeps track of the active time of each zone valve.

```

1 void loop() {
2   // LCD button taps detected
3   detectTaps();
4
5   // Different button taps
6   switch (buttonStatus)
7   {
8     case ZERO_TAP: // v
9       if (controlLCD ==
10         ValvesStatusLCD(
11         controlLCD = 0;
12       }
13
14       if (showTimeLCD ==
15         ValvesTimersLCD(
16       }
17
18       break;
19
20     case SINGLE_TAP: //
21       Serial.println("Si
22       vals.z1_local = !\
23       sendValues(&vals);
24       buttonStatus = ZEF
25       break;
26
27     case DOUBLE_TAP: //
28       Serial.println("De
29       vals.z2_local = !\

```

Arduino MKR WiFi 1010

Code

The MKR WiFi 1010 needs the following libraries:

`ArduinoIoTCloud.h` handles the Arduino Cloud connection and project variables publishing. It can be installed directly from the Arduino Library Manager.

`Arduino_ConnectionHandler.h`

manages the Wi-Fi® connection and can be installed directly from the Arduino Library Manager.

`ArduinoJson.h` and

`Arduino_JSON` parse and create JSON structures for the HTTP requests. They can be

`ArduinoHttpClient.h`

requests weather data from the Open Weather API. It can be installed directly from the Arduino Library Manager.

`Wire.h`

will enable the I2C communication between the Edge Control, the MKR WiFi 1010 and the other peripherals. It is included in the BSP of the MKR WiFi board.

`utility/wifi_drv.h`

controls the MKR built-in RGB LED. It is included in the BSP of the MKR WiFi 1010.

There are two headers included in the project code that handles some helper functions and structures:

`thingProperties.h` is

automatically generated by the Arduino Cloud. However, if you are using an offline IDE, verify it is in the same directory as your sketch and includes all the Arduino Cloud variables.

`SensorValues.hpp` handles

the shared variables between the Edge Control and the MKR WiFi 1010 through I2C.

In the global variables, the MKR board I2C address is defined. It must be the same as defined in the Edge Control code. In addition, the water tank dimensions are also set up.

```

1  #include "thingProperties
2  #include <ArduinoJson.h>
3  #include <Wire.h>
4  #include "SensorValues.hp
5  #include <utility/wifi_dr
6  #include <ArduinoHttpClie
7  #include <Arduino_JSON.h>
8
9  // The MKR1 board I2C add
10 #define SELF_I2C_ADDR 0x0
11
12 // Water tank constants
13 #define PI 3.141592653589
14 #define radius 0.28 // i

```

To properly retrieve weather information from the [Open Weather API](#), you must create an account to use it; there you will find a unique API Key needed to format your HTTP requests.

Replace the `city` and `countryCode` variables with yours.

```

1  // OpenWeather server add
2  const char serverAddress[
3  int port = 443;
4
5  // Your OpenWeather API K
6  String openWeatherMapApiK
7
8  // Replace with your coun
9  String city = "Santiago d
10 String countryCode = "DO"

```

The MKR WiFi 1010 communicates with the Arduino Cloud and checks for changes on every variable, from the valves' switches to the scheduled activated timers. The board turns its LED blue when it is successfully connected to the Cloud. Also, it asks

humidity and rain probability in the Arduino Cloud dashboard.

```

1 void loop() {
2   // function that asks
3   scheduleHandler();
4
5   unsigned long currentM
6
7   if (currentMillis - pr
8       // save the last tim
9       previousMillis = cur
10
11   // Turn on the blue
12   if (ArduinoCloud.cor
13       WiFiDrv::analogWri
14   } else {
15       WiFiDrv::analogWri
16   }
17 }
18
19 // doing this just once
20 if (bootForecast == 1
21     getForecast(); // r
22     getWeather(); // r
23     bootForecast = 0;
24 }
25
26
27 if (millis() - lastRec
28     // request weather \

```

The system needs to be as fast as possible when a valve activation is requested. For this reason, each valve has its own function that is called once the valve status changes on the Arduino Cloud.

```

1  /*
2   Since Z1 is READ_WRITE
3   executed every time a n
4  */
5  void onZ1Change() {
6   // Add your code here t
7   Serial.println("Z1 chan
8   SensorValues_t vals;
9   vals.z1_local = z1;
10  delay(1000);
11  }

```

The

```
uploadValues(SensorValues_t
*vals)
```

is executed every time the Edge Control requests data to the MKR through I2C communication. It updates all the local and Cloud-shared variables to be sent and converts the water level variable to water volume applying the cylinder volume formula:

$$v = \pi r^2 h$$

Where:

r is the tank radius in meters.

h is the tank height in meters.

As r and h are in meters, the volume result will be in cubic meters (m^3). For a more intuitive perception of water quantity, the volume can be converted into liters:

$$1m^3 = 1000 \text{ L}$$

Since the level sensor output is in centimeters, it can be converted (height) into meters by dividing by 100: $h = (\text{water_level}/100)$

```
water_volume = PI*(radius)*  
(radius)*(water_level/100)*1000
```

To evaluate water consumption, the initial water level is stored as a reference point. Any level measured below that point will be considered as consumed water, updating the `water_usage` variable. If the tank is refilled, the new water level will become the new reference point.

```
1  water_level = vals->water  
2  water_volume = PI * (ra  
3  
4  if (waterCtrl == 0) {  
5    currentWater = water_  
6    Serial.println("Initi  
7    Serial.println(curren  
8    water_usage = 0;  
9    waterCtrl = 1;  
10 }  
11  
12 if (water_volume < curr  
13   water_usage += curren  
14   currentWater = water_  
15 } else if (water_volume  
16   currentWater = water_  
17 }
```

The Arduino Cloud Dashboard

Taking advantage of the Arduino Cloud, it is possible to seamlessly integrate a simple but powerful dashboard to monitor and visualize the status of the system from remote, resulting in a professional Human-Computer Interaction (HCI) as shown below:



Arduino Cloud project dashboard

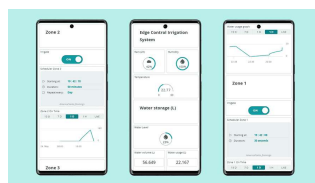
Within the Arduino Cloud's dashboard, the system variables can be monitored and controlled as follow:

Temperature, humidity and rain probability are displayed in gauges, showing the current weather status and forecast.

Each motorized valve can be toggled through a dedicated switch accompanied by a scheduler widget to set automatic irrigation routines.

Each valve activated time is shown in a time series chart.

Water consumption can be monitored in dedicated widgets: one that shows the current water level in percentage from 0-100%, a water volume widget that shows the remaining liters of water in the tank and a water usage widget that shows the daily consumed liters.



Arduino Cloud project dashboard on a smartphone

The dashboard is easily accessible from a browser, mobile phone or tablet, allowing a user to receive an instantaneous update on the irrigation status from anywhere.

Full Smart Irrigation System Example

All the necessary files to replicate this application note can be found below:

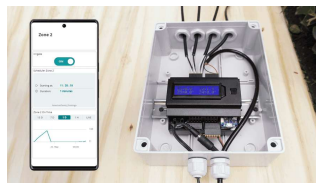
The complete code can be downloaded [here](#).

The Irrigation System Working

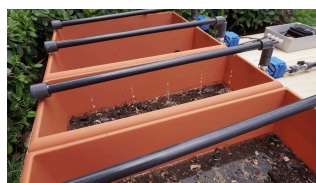
Below you can find some additional images and animations showing how the system works:



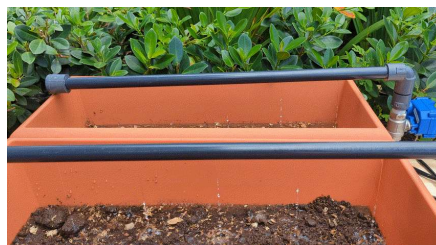
Zone one manual irrigation



Automatic irrigation timer countdown in zone two
(1.4 minutes left)



Zone two after the motorized valve is opened



Conclusion

In this application note, you have learned how to build a smart irrigation system to water your crops automatically, manually or remotely. Thanks to real-time weather analysis, you can avoid irrigation when it is raining, saving water and avoiding over-irrigation or flooding problems.

Arduino Edge Control allows you to easily implement this kind of agriculture systems ready for field deployment. However, if Wi-Fi® access is limited in your area, the connectivity can be extended using alternative boards, like the [Arduino MKR WAN 1310](#).

Thanks to the Edge Control capabilities to connect to the Cloud and be remotely controlled, it is a great choice for developing robust and agriculture environment-proof solutions.

Next Steps

Since you already know how to develop a Smart Irrigation System with Arduino Edge Control and the MKR WiFi 1010, it is time for you to continue exploring all the capabilities of the Arduino Pro portfolio and integrating it into your professional setup.

You can extend the capabilities of your Edge Control-based system by adding different connectivity options, leveraging the [Arduino MKR family](#) like LoRa® technology, GSM, RS-485 or Ethernet.

Trademark

Acknowledgments

LoRa® is a registered trademark of Semtech Corporation.

Suggest changes

The content on [docs.arduino.cc](#) is facilitated through a public [GitHub repository](#). If you see anything wrong, you can edit this page [here](#).

Need support?

[Help Center](#)
[Ask the Arduino Forum](#)
[Discover Arduino Discord](#)

License

The Arduino documentation is licensed under the [Creative Commons Attribution-Share Alike 4.0](#) license.

Was this article helpful?

